

Архитектура текущего логирования (результат 8.1 + 8.2)

Документ фиксирует текущее состояние логирования в проекте LaravelExample: что и где логируется, как передается контекст, как связаны логи и Sentry.

1) Что пришло из уроков 8.1 и 8.2

Из урока 8.1 в проекте закреплены практики:

- уровни логирования (`info`, `warning`, `error`);
- структурированный контекст в `Log::*`;
- `correlation_id` как ключ трассировки запроса.

Из урока 8.2 внедрено:

- интеграция Sentry через `sentry/sentry-laravel`;
- проброс контекста в Sentry scope;
- отдельный `PaymentErrorReporter` для единого репортинга payment-ошибок (лог + Sentry).

2) Текущая конфигурация каналов логирования

Фактически сейчас (по `.env` и `config/logging.php`):

- `LOG_CHANNEL=stack`
- `LOG_STACK=single,sentry_logs`
- `LOG_LEVEL=info`

Что это означает:

- логи Laravel пишутся в файл `storage/logs/laravel.log` (канал `single`);
- логи уровня от `info` и выше дополнительно могут уходить в канал `sentry_logs` (в зависимости от конфигурации Sentry);
- по умолчанию это не `stdout`-режим контейнера.

Для live-демо 8.3 можно временно переключать на `stderr` (чтобы видеть в `docker compose logs`).

3) Архитектурная схема потока логов

```
HTTP request / Job
-> AssignCorrelationId middleware (HTTP)
  - генерирует/принимает X-Correlation-ID
  - заполняет Context (correlation_id, endpoint, method, path,
  user_id)
  - Log::shareContext(Context::all())
  - добавляет X-Correlation-ID в response
-> Controller / Service / Job / Listener
  - Log::info|warning|error с доменным контекстом
```

- в payment errors: `PaymentErrorReporter`
- > Monolog channels (stack => single + sentry_logs)
 - single: `storage/logs/laravel.log`
 - sentry_logs: Sentry logs channel
- > Sentry
 - исключения (глобальная интеграция + custom capture)
 - tags/extra/user + correlation_id

4) Где уже покрыто логированием

4.1 HTTP/API слой

- `AssignCorrelationId`:
 - выставляет `X-Correlation-ID`;
 - кладет request-метаданные в `Context`;
 - делится контекстом с логером через `Log::shareContext(...)`;
 - добавляет `correlation_id` и `endpoint` в Sentry scope.
- `PaymentController`:
 - при ошибках создания платежа вызывает `PaymentErrorReporter`;
 - есть demo endpoint `POST /api/v1/payments/demo-fail` для controlled проверки observability.

4.2 Service слой

- `PaymentService`:
 - `Log::info('Payment created', ...)`;
 - `Log::info('Payment deleted', ...)`.
- `AccountService`:
 - `Log::info('Account created', ...)`;
 - `Log::info('Account deleted', ...)`.
- `PaymentErrorReporter`:
 - `Log::error('Payment processing failed at gateway level', ...)`;
 - объединяет доменный контекст ошибки с `Context::all()`;
 - отправляет событие в Sentry с tags/user/extra.

4.3 Queue/Jobs/Listeners

- Jobs:
 - `SendPaymentConfirmationJob`: warning, если payment не найден;
 - `RobustSendPaymentConfirmationJob`: warning (not found), info (skip already sent), error (mail fail);
 - `CheckPaymentStatusesJob`: info при выполнении;
 - `DailyPaymentsReportJob`: info при выполнении;
 - `LogPaymentToAuditJob`: warning, если payment не найден.
- Listener:
 - `SendPaymentConfirmationNotificationListener`: warning, если user не найден.

5) Что уже покрыто с точки зрения observability

Уже есть:

- сквозной `correlation_id` для API-запросов;
- структурированные контекстные логи по ключевым сущностям (`payment_id`, `account_id`, `user_id`, `gateway_code`);
- централизованный репортинг payment-failure в Sentry;
- demo-endpoints для учебной демонстрации и проверки интеграции.

Частично покрыто (зона развития):

- единые naming-правила сообщений и полей для всех jobs/listeners;
- расширение контекста для аsync-цепочек (чтобы correlation/request context последовательно жил в очередях);
- явная матрица событий `info/warning/error/critical` для всех критичных бизнес-операций.

6) Как проверить текущее состояние (через docker compose)

Проверка роутов и тестов:

```
docker compose ps
docker compose exec php php artisan route:list --path=api/v1
f
```

Проверка payment error flow:

```
curl -i -X POST http://localhost/api/v1/payments/demo-fail \
-H "Accept: application/json" \
-H "X-Correlation-ID: arch-check-001"
```

Проверка логов (дефолтный файловый режим):

```
docker compose exec php sh -lc "tail -n 100 storage/logs/laravel.log"
```

Проверка Sentry integration:

```
curl -i http://localhost/api/v1/test-sentry -H "X-Correlation-ID: arch-sentry-001"
```

7) Короткий вывод

Текущая архитектура уже соответствует учебной цели модулей 8.1 и 8.2:

- базовые практики структурированного логирования внедрены;

- `correlation_id` и контекст централизованы;
- Sentry интегрирован как error-tracking слой;
- проект готов к уроку 8.3, где акцент на лог-агрегацию и incident workflow.